SOLVING PROBLEMS BY SEARCHING

One kind of goal-based agent called a problem-solving agent.

Uninformed search algorithms that are **given no information** about the problem other than its definition. Although some of these algorithms can solve any solvable problem, none of them can do so efficiently. Informed search algorithms, on the other hand, can do quite well given some guidance on where to look for solutions.

Imagine an agent in the city of Arad, Romania, enjoying a touring holiday. The agent's performance measure contains many factors: it wants to improve its suntan, improve its Romanian, take in the sights, enjoy the nightlife, and avoid hangovers, and so on.

Let to go Bucharest, there is too much uncertainty in the world and there would be too many steps in a solution. Problem formulation is the process of deciding what actions and states to consider, given a goal. For now, let us assume that the agent will consider actions at the level of driving from one major town to another. Our agent has now adopted the goal of driving to Bucharest and is considering where to go from Arad.

Three roads lead out of Arad, one toward Sibiu, one to Timisoara, and one to Zerind. None of these achieves the goal, so unless the agent is familiar with the geography of Romania, it will not know which road to follow.' In other words, the agent will not know which of its possible actions is best, because it does not yet know enough about the state that results from taking each action. If the agent has no additional information—i.e., if the environment is unknown then it is has no choice but to try one of the actions at random.

But suppose the agent has a map of Romania. The point of a map is to provide the agent with information about the states it might get itself into and the actions it can take. The agent can use this information to consider subsequent stages of a hypothetical journey via each of the three towns, trying to find a journey that eventually gets to Bucharest.

Once it has found a path on the map from Arad to Bucharest, it can achieve its goal by carrying cut the driving actions that correspond to the legs of the journey. In general, an agent with several immediate options of unknown value can decide what to do by first examining future actions that eventually lead to states of known value.

The process of looking for a sequence of actions that reaches the goal is called **search.**

A problem can be defined formally by five components:
Basic Components of a search problem:
->Initial State
->A description of the possible actions available to the agent.
-> Transition model / successor function
->Goal Test
->Path Cost

The **initial** state that the agent starts from that. For example, the initial state for our agent in Romania might be described as *In(Arad).*

A description of the possible actions available to the agent: Given a particular state s, ACTIONS(s) returns the set of actions that can be executed in s. We say that each of these actions is **applicable** in s. For example, from the state *In(Arad),* the applicable actions are { *Go(Sibiu), Go(Timisoara), Go(Zerind)*}.

A description of what each action does; the formal name for this is the **transition model,** specified by a function Result(s, a) that returns the state that results from doing action a in state *s.* We also use the term **successor** to refer to any state reachable from a given state by a single action. For example, we have RESULT(In(Arad), *Go(Zerind)) = In(Zerind)* .

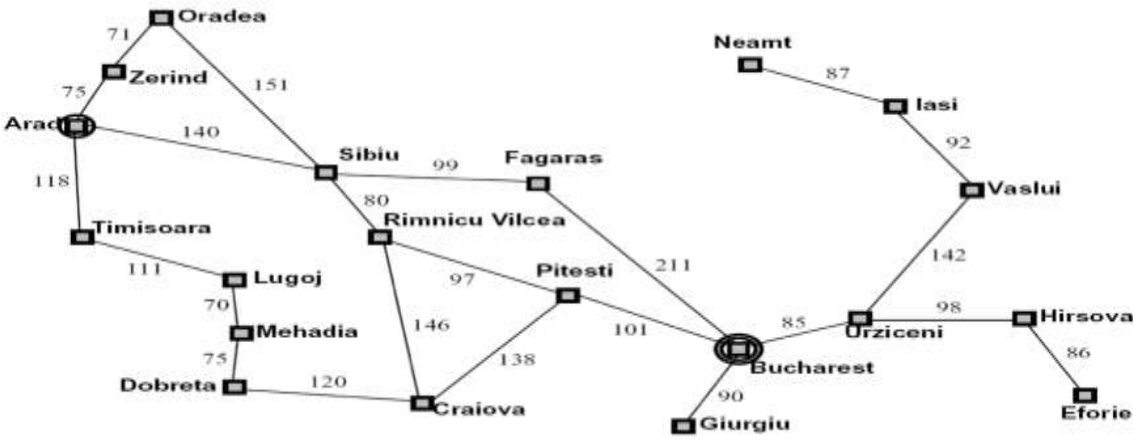
Together, the initial state, actions, and transition model implicitly define the **state space** of the problem—the set of all states reachable from the initial state by any sequence of actions. A **path** in the slate space is a sequence of states connected by a sequence of actions.

The **goal test,** which determines whether a given state is a goal state. Sometimes there is an clear set of possible goal states, and the test simply checks whether the given state is one of them. The agent's goal in Romania is the singleton set { *In(Bucharest)*}. For example, in chess, the goal is to reach a state called "checkmate," where the opponent's king is under attack and can't escape.

A path cost function that assigns a numeric cost to each path. The problem-solving agent chooses a cost function that reflects its own performance measure. For the agent trying to get to Bucharest, time is of the essence, so the cost of a path might be its length in kilometers.

A solution is a sequence of actions (a plan) which transforms the start state to a goal state. Solution quality is measured by the path cost function, and an optimal solution has the lowest path cost among all solutions.

Travelling in Romania:



->Initial State: Arad
->Actions: Take adjacent Roads
->Transition model: Roads take to the next city
->Goal Test: Bucharest
->Path Cost: mileage

Toy problems

The first example we examine is the vacuum world. This can be formulated as a problem as follows:

-> States: The state is determined by both the agent location and the dirt locations. The agent is in one of two locations, each of which might or might not contain dirt. Thus, there are 2 x 2^2 = B possible world states. A larger environment with n locations has n • 2^n states.
-> Initial state: Any state can be designated as the initial state.
-> Actions: In this simple environment, each state has just three actions: Left, Right, and Suck. larger environments might also include Up and Down.
-> Transition model: The actions have their expected effects, except that moving Left in the leftmost square, moving Right in the rightmost square, and Sucking in a clean square have no effect. The complete state space is shown in Figure 3.3.
-> Goal test: This checks whether all the squares are clean.
-> Path cost: Each step costs 1, so the path cost is the number of steps in the path.

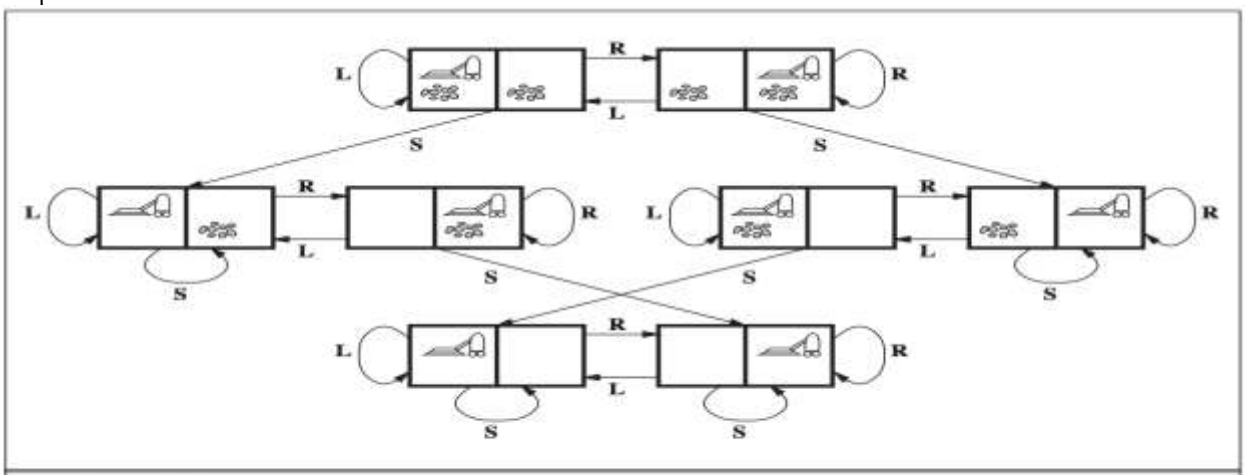


Figure 3.3 The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

8-puzzle problem:

The 8-puzzle, an instance of which is shown in Figure 3.4, consists of a 3 x3 board with eight numbered tiles and a blank space. A tile adjacent to the blank space can slide into the space. The object is to reach a specified goal state, such as the one shown on the tight of the figure. The standard formulation is as follows:

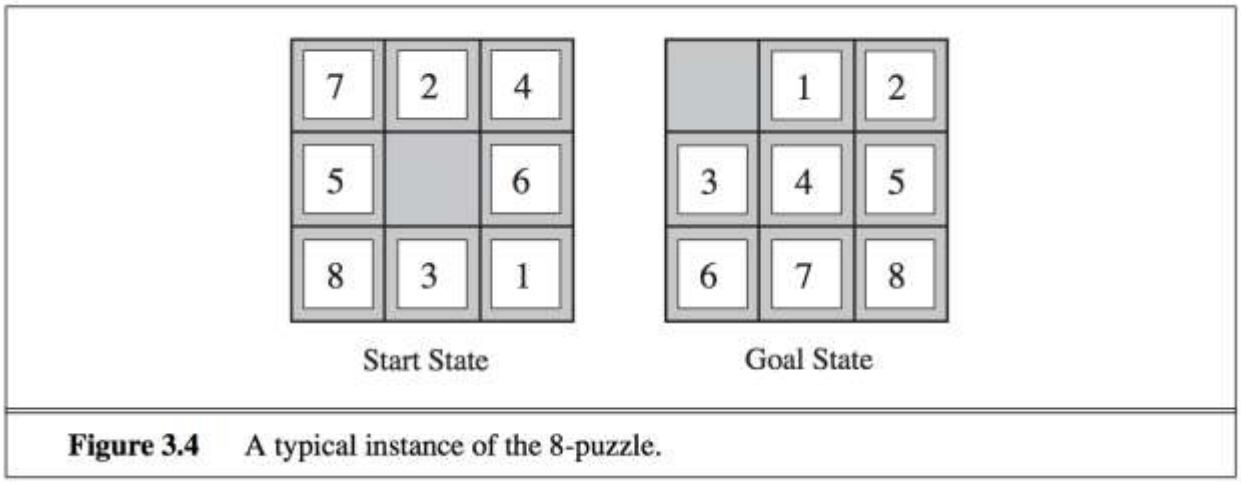


Figure 3.4 A typical instance of the 8-puzzle.

->States: A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
->Initial state: Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states (Exercise 3.4)
->Actions: The simplest formulation defines the actions as movements of the blank space Left, Right, Up, or Down. Different subsets of these are possible depending on where the blank is.
->Transition model: Given a state and action, this returns the resulting state; for example, if we apply Left to the start state in Figure 3.4, the resulting state has the 5 and the blank switched.
->Goal test: This checks whether the state matches the goal configuration shown in Figure 3.4. (Other goal configurations are possible.)
->Path cost: Each step costs 1, so the path cost is the number of steps in the path.

8-queens problem:

The goal of the 8-queens problem is to place eight queens on a chessboard such that no queen attacks any other. (A queen attacks any piece in the same row, column or diago-nal.)

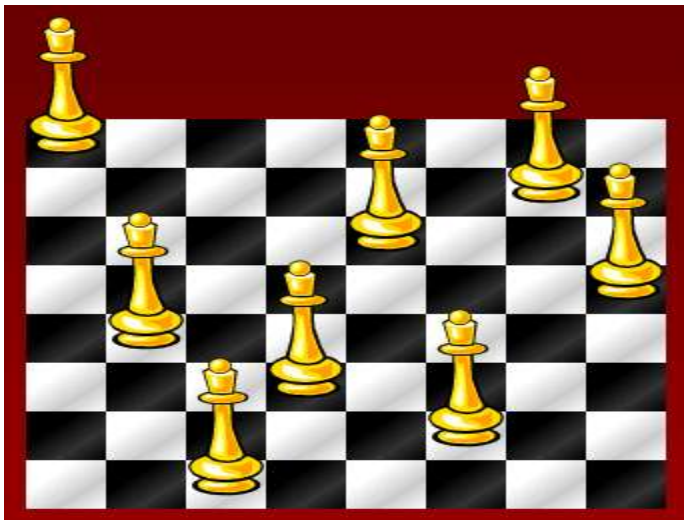


Figure 3.5 Almost a solution to the 8 queens problem.

There are two main kinds of formulation. An incremental formulation involves operators that enlarge the state description, starting with an empty state; for the 8-queens problem, this means that each action adds a queen to the state. A complete-state formulation starts with all 8 queens on the board and moves them amend. In either case, the path cost is of no interest because only the final state counts. The first incremental formulation one might try is the following:
-> States: Any arrangement of 0 to 8 queens on the board is a state.
->Initial state: No queens on the board.
-> Actions: Add a queen to any empty square.
-> Transition model: Returns the board with a queen added to the specified square.
-> Goal test: 8 queens are on the board, none attacked.

Measuring problem-solving performance

We can evaluate an algorithm's performance in four ways:

->Completeness: Is the algorithm guaranteed to find a solution when there is one?
->Optimality: Does the strategy find the optimal solution?
->Time complexity: How long does it take to find a solution?
->Space complexity: How much memory is needed to perform the search?

UNINFORMED SEARCH STRATEGIES:

Uninformed search (blind search) has no information about the number of steps or the path costs from the current state to the goal. They can only distinguish a goal state from a non-goal state. There is no bias to go towards the desired goal.

Uninformed search the term means that the strategies have no additional information about states beyond that provided in the problem definition. All they can do is generate successors and distinguish a goal state from a non-goal state. All search strategies are distinguished by the order in which nodes are expanded. Strategies that know whether one non-goal state is "mom promising" than another arc called informed search or heuristic search strategies.
An uninformed search algorithm generates the search tree without using any domain specific knowledge.

Example of uninformed search algorithm:

->Uniform-cost search
->Breadth-First-Search
->Depth-First-Search
->Depth-Limited-Search
->Depth-First Iterative Deepening
->Bi-Directional Search

•Uninformed search methods have access only to the problem definition. The basic algorithms are as follows:

—**Breadth-first** search expands the shallowest nodes first; it is complete, optimal for unit step costs but has exponential space complexity.

—Uniform cost search expands the node with lowest path cast, g(n), and is optimal for general step costs.

—**Depth-first search** expands the deepest unexpanded node first. It is neither com-plete nor optimal, but has linear space complexity. **Depth limited search** adds a depth bound.

—Iterative deepening search calls depth-first search with increasing depth limits until a goal is found. It is complete, optimal for -unit step costs, has time complexity comparable to breadth-first search, and has linear space complexity.

—**Bidirectional search** can enormously reduce time complexity, but it is not always applicable and may require too much space.

Uniform-cost search:

Uniform-cost search expands the node n with the lowest path cost y(n). This is done by storing the frontier as a priority queue ordered by q. The example shown in Figure 3.15, where the problem is to get from Sibiu to Bucharest. The successors of Sibiu are Rimnicu Vilcea and Fagaras, with costs 80 and 99, respectively. The least-cost node, Rimnicu Vilcea, is expanded next, adding Pitesti with cost 80 + 97 =177. The least-cost node is now Fagaras, so it is expanded, adding Bucharest with cost 99 + 211 = 3111. Now a goal node has been generated; but uniform-cost search keeps going, choosing Pitesti for expansion and adding a second path to Bucharest with cost 80+97+101= 278. Now the algorithm checks to see if this new path is better than the old one: it is, so the old one is discarded. Bucharest, now with g-cost 278, is selected for expansion and the solution is returned.

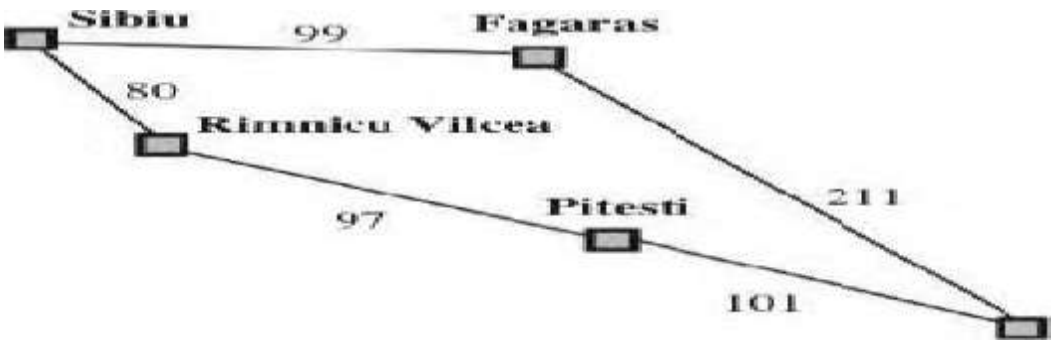
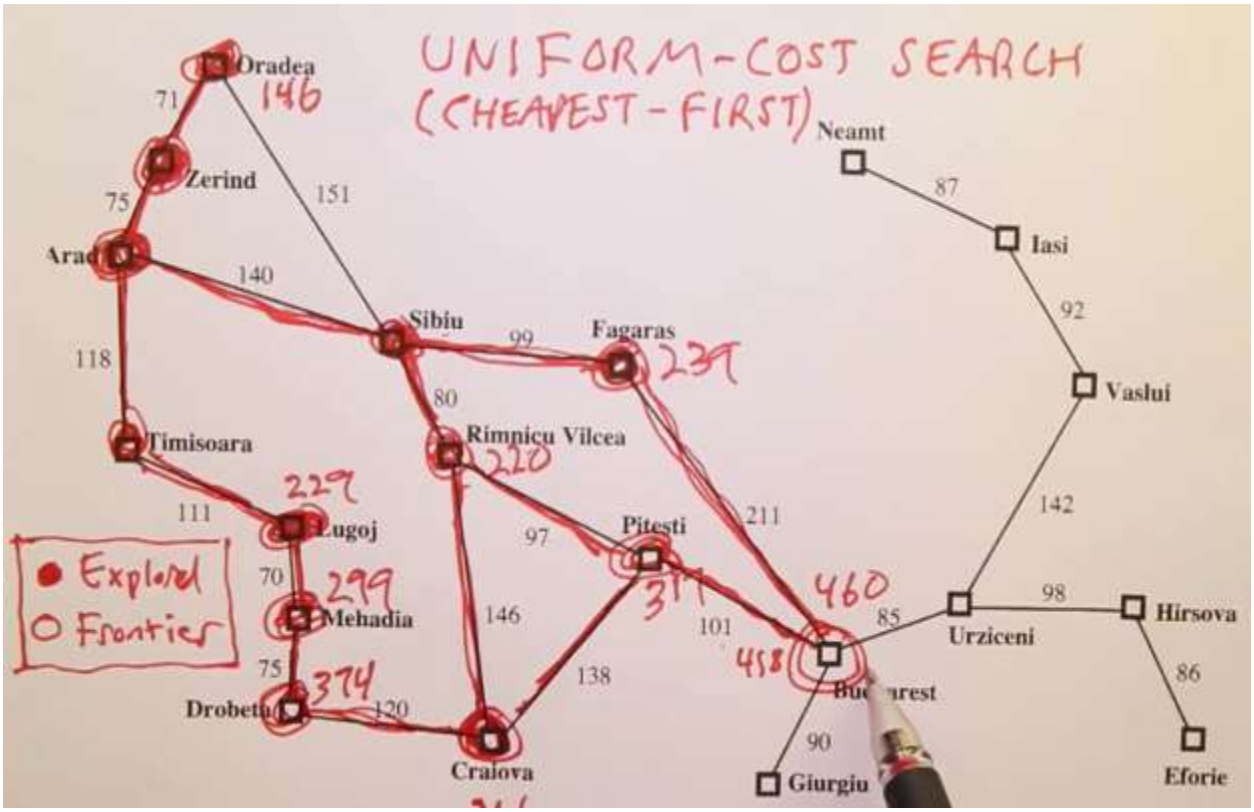


Figure 3.15 Part of the Romania state space, selected to illustrate uniform-cost search.

The problem is to get from Arad to Bucharest:

What is the solution?



Uniform-cost search

- Store node and cost to get to node in queue
- Use a priority queue instead of a standard queue
- Always choose the cheapeast node to expand
- "Expand" means examine children of node

-> Uniform-Cost Pseudocode

```
enqueue(initialState)
do
node = prioroty-dequeue()
if (node not in closed list)
      add node to closed list
if goalTest(node)
    return node (potenially path as well)
else
   children = successors(node)
   for child in children
        prioroty-enqueue(child, dist(child))
```

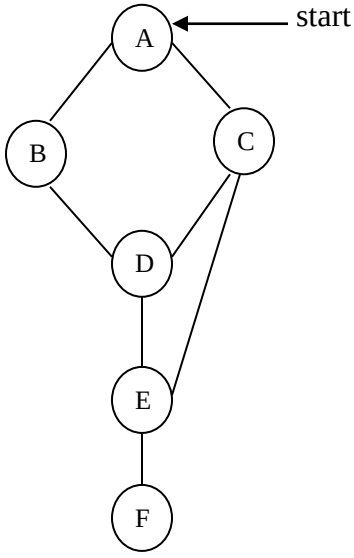
Note: dist is the cost of the path from the initial state to the child node

Breadth First Search (BFS)

Algorithm of BFS:

```
queue<=start
while(queue!=empty)
  v<=queue
    if (v is not visited)
      visit v
      visit sequence<=v
         for(all non-visited adjacent u of v)
           queue<=u
         end for
    end if
end while
```

Mechanism:



queue

~~A~~			

v<=queue	visit v	visited sequence	u=non visited-adjacent vertices u of v	queue<=u
v='A'	A	A	u=B, C	~~B~~ C
v='B'	B	AB	u=~~A~~, ~~D~~	C D
v='C'	C	ABC	u=~~A~~, ~~D~~, E	D D E
v='D'	D	ABCD	u= ~~B~~, ~~C~~, ~~E~~	D E E

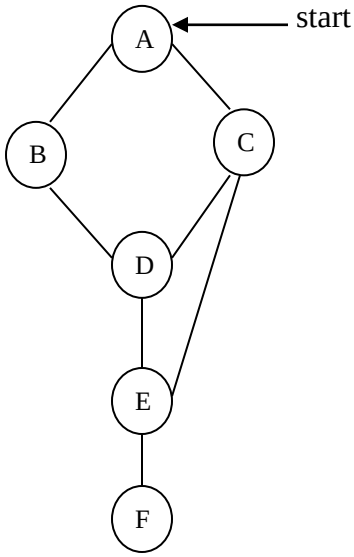
v=’D’	×	ABCD	×	E E
v=’E’	E	ABCDE	u= D, F	E F
v=’E’	×	ABCDE	×	F
v=’F’	F	ABCDEF	u= E	

Depth First Search (DFS)

Algorithm of DFS:

```
stack<=start
while(stack!=empty)
  v<=stack
    if (v is not visited)
       visit v
       visit sequence<=v
      for(all non-visited adjacent u of v)
          stack<=u
      end for
    end if
end while
```

Mechanism



stack

A			

v<=stack	visit v	visited sequence	u=non visited-adjacent vertices u of v	stack<=u
v='A'	A	A	u=B, C	B ~~C~~
v='C'	C	AC	u=A, ~~D~~,E	B D E
v='E'	E	ACE	u= C, ~~D~~, F	B D D F
v='F'	F	ACEF	u= ~~E~~	B D D
v='D'	D	ACEFD	u= B, C, ~~E~~	B D B
v='B'	B	ACEFDB	u= A, ~~D~~	B D
v='D'	×	ACEFDB	×	B
v='B'	×	ACEFDB	×	

BFS:

```
#include<stdio.h>
#include<conio.h>
char que[20];
int front=0, rear=0, n;
char arr[20];
int bfs(int );
char ajMat[20][20];
char b[20];
void display();
int p=0;
int main()
{
   char v;
          int i,j;
   printf("Enter the number of nodes in a graph:=");
   scanf("%d",&n);
   printf("Enter the value of node of graph:=");
   for( i=0; i<n; i++)
   {
      scanf("%s",&b[i]);
   }
   printf("Enter the value in adjancency matrix in from of 'y' or 'n':=\n");
   printf("If there exits an edge between two vertices than 'y' otherwise 'n':=\n");
   for( i=0; i<n; i++)
   printf(" %c",b[i]);
   for( i=0;i<n; i++)
   {
```

```
ss        printf("\n%c ",b[i]);
        for( j=0; j<n; j++)
        {
            printf("%c ",v=getch());
            ajMat[i][j]=v;
        }
        printf("\n\n");
    }
    for( i=0;i<n;i++)
    bfs(i);

    display();
    getch();
            return 0;
}
void display()
{
     printf("BFS of Graph : ");
     for(int i=0; i<n; i++)
     printf("%c ",arr[i]);
}
void insert(char val)
{
    que[front]=val;
    front++;
}

char del()
{
    rear=rear+1;
    return que[rear-1];
}
bool unVisit(char val)
{
    for(int i=0; i<front; i++)
    {
        if(val==que[i])
        return false;
    }
return true;
}

int bfs(int i)
{
     char m;
     if(front==0)
     {
         insert(b[i]);
     }
     for(int j=0; j<n; j++)
     {
```

```
        if(ajMat[i][j]=='y')
        {
            if(unVisit(b[j]))
            {
                insert(b[j]);
            }
        }
    }
    m=del();
    arr[p]=m;
    p++;
    return 0;
}
```

Output:

Enter the number of nodes in a graph:=6
Enter the value of node of graph:=a b c d e f
Enter the value in adjacency matrix in from of 'Y' or 'N':
If there is an edge between the two vertices then enter 'Y' or 'N':=

	a	b	c	d	e	f
a	n	y	y	n	n	n
b	y	n	n	y	n	n
c	y	n	n	y	y	n
d	n	y	y	n	y	n
e	n	n	y	y	n	y
f	n	n	n	n	y	n

BFS of graph: a b c d e f

DFS:

```
#include<stdio.h>
#include<conio.h>
char stack[20];
int top=-1, n;
char arr[20];
char dfs(int );
char ajMat[20][20];
char b[20];
void display();
int p=0,i;
int main()
{
   char v;
   int l=0;
   printf("Enter the number of nodes in a graph:=");
   scanf("%d",&n);
   printf("Enter the value of node of graph:=");
```

```
    for(i=0; i<n; i++)
    {
       scanf("%s",&b[i]);
    }
    char k=b[0];
    printf("Enter the value in adjancency matrix in from of 'Y' or 'N':\n");
    printf("\nIf there is an edge between the two vertices then enter 'Y' or 'N':=\n");
    for(i=0; i<n; i++)
    printf(" %c ",b[i]);
    for(i=0;i<n; i++)
    {
        printf("\n%c ",b[i]);
        for(int j=0; j<n; j++)
        {
            printf("%c ",v=getch());
            ajMat[i][j]=v;
        }
        printf("\n\n");
    }
    for(i=0;i<n;i++)
    {
        l=0;
        while(k!=b[l])
        l++;
        k=dfs(l);
    }
    display();
    getch();
           return 0;
}

void display()
{
     printf(" DFS of Graph : ");
     for(int i=0; i<n; i++)
     printf("%c ",arr[i]);
}
void push(char val)
{
     top=top+1;
     stack[top]=val;
}
char pop()
{
     return stack[top];
}
bool unVisit(char val)
{
      for(i=0; i<p; i++)
      if(val==arr[i])
      return false;
```

```
    for(i=0; i<=top; i++)
    if(val==stack[top])
    return false;
    return true;
}
char dfs(int i)
{
     char m;
   if(top==-1)
   {
       push(b[i]);
   }
   m=pop();
   top--;
   arr[p]=m;
   p++;
   for(int j=0; j<n; j++)
   {
       if(ajMat[i][j]=='y')
       {
           if(unVisit(b[j]))
           {
               push(b[j]);
           }
       }
   }
  return stack[top];
}
```

Output:

Enter the number of nodes in a graph:=6
Enter the value of node of graph:=a b c d e f
Enter the value in adjacency matrix in from of 'Y' or 'N':
If there is an edge between the two vertices then enter 'Y' or 'N':=

	a	b	c	d	e	f
a	n	y	y	n	n	n
b	y	n	n	y	n	n
c	y	n	n	y	y	n
d	n	y	y	n	y	n
e	n	n	y	y	n	y
f	n	n	n	n	y	n

DFS of graph: a c e f d b

Depth-First Iterative Deepening Search:

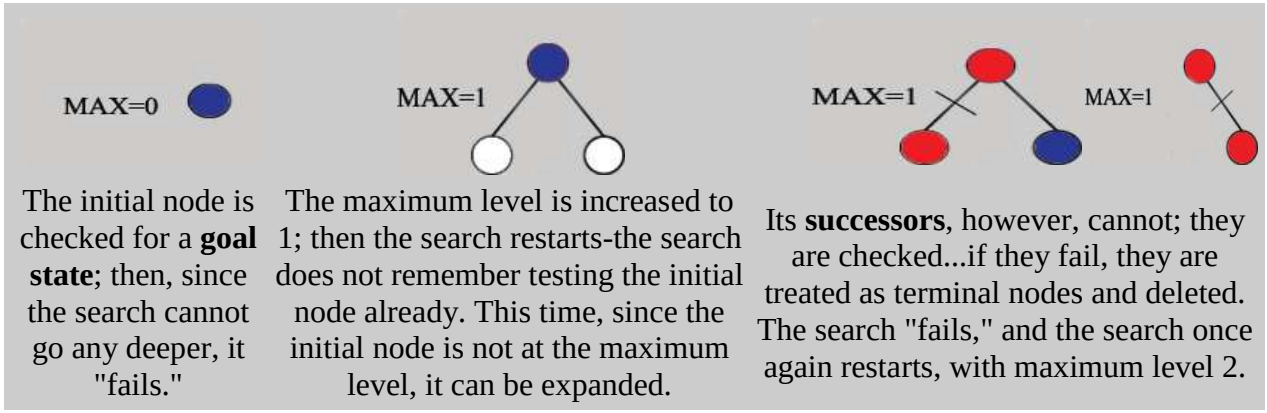
In depth-first search, you explore each branch you enter completely before backtracking from it and going to the next one. In iterative deepening, you don't go below the current depth, and hence don't explore each branch you visit completely before backtracking.

While still an unintelligent algorithm, the iterative deepening search combines the positive elements of breadth-first and depth-first searching to create an algorithm which is often an improvement over each method individually.

An iterative deepening search operates like a depth-first search, except slightly more constrained--there is a maximum depth which defines how many levels deep the algorithm can look for solutions. A node at the maximum level of depth is treated as terminal, even if it would ordinarily have successor nodes. If a search "fails," then the maximum level is increased by one and the process repeats. The value for the maximum depth is initially set at 0 (i.e., only the initial node).

Visited Nodes

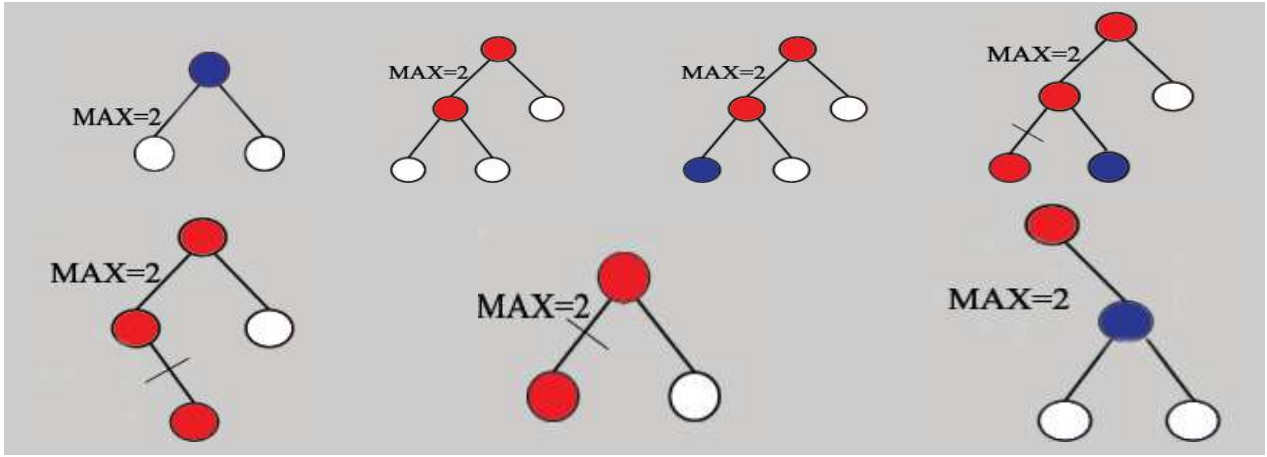
Current Node



The initial node is checked for a **goal state**; then, since the search cannot go any deeper, it "fails."

The maximum level is increased to 1; then the search restarts-the search does not remember testing the initial node already. This time, since the initial node is not at the maximum level, it can be expanded.

Its **successors**, however, cannot; they are checked...if they fail, they are treated as terminal nodes and deleted. The search "fails," and the search once again restarts, with maximum level 2.

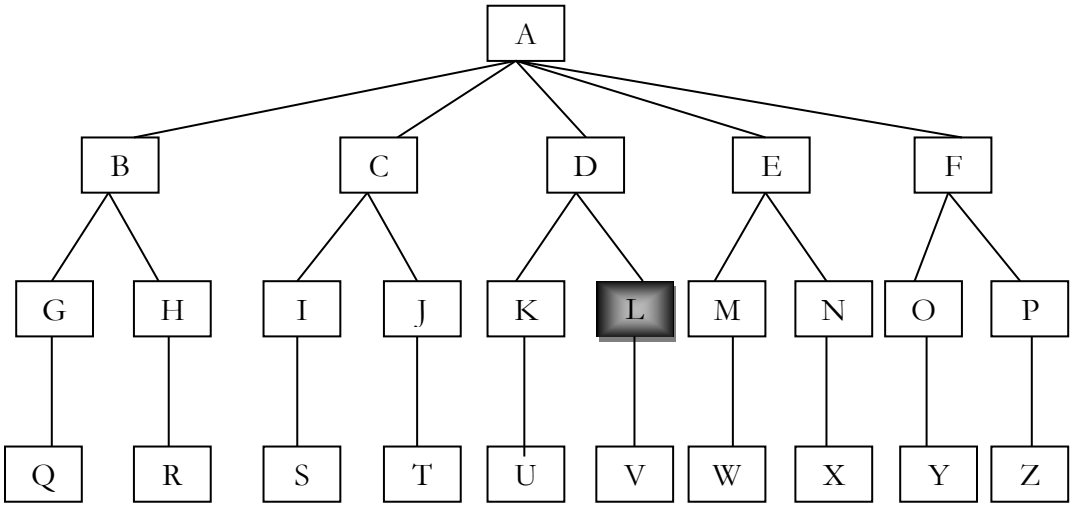


This continues until a solution is found.

An interesting observation is that the nodes in this search are first checked in the same order they would be checked in a breadth-first-search; however, since nodes are deleted as the search progresses, much less memory is used at any given time.
The drawback to the iterative deepening search is clear from the walkthrough--it can be painfully redundant, rechecking every node it has already checked with each new iteration. The algorithm can be enhanced to remember what nodes it has already seen, but this sacrifices most of the memory efficiency that made the algorithm worthwhile in the first place, and nodes at the maximum level for one iteration will still need to be re-accessed and expanded in the following iteration. Still, when memory is at a premium, iterative deepening is preferable to a plain depth-first search when there is danger of looping or the most efficient solution is desired.

Mechanism of Depth-First Iterative deepening Search:

->The problem with depth limited search is deciding on a suitable depth parameter .To avoid this problem there is another search called Depth–First Iterative Deepening search.
->This search method tries all possible limits; first 0, then 1 , then2 etc., until a solution is found .
->DFIDS may seem wasteful as it is expanding nodes multiple times. But the overhead is small in comparison to the growth of an exponential search tree.
->For large search spaces where is the depth of the solution is not known IDS is normally preferred search method.
->The following is the example an IDS of 26 nodes (states) with an initial state of node A and a goal state of node L.





=visited node



= expanded node

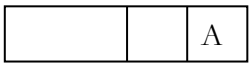


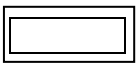
=Goal node

0th iteration:



We begin with our initial state: the node labeled A. This node is added to the stack. Node A is then removed from the stack and visited.

Stack 

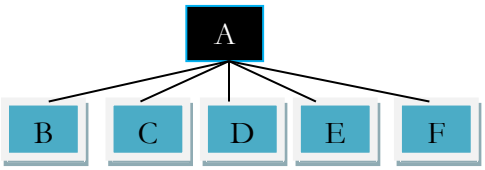
v<=stack	action	visit (v)	visited sequence	u=non-visited adjacent u of v	goal test	stack update	no. of level	no. of visited nodes
v='A'	visit	A	A	n/a	is A==goal?		0	1

As the stack is empty, but the goal state is not found.
So we need to begin 1st iteration.

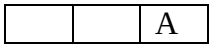
1st iteration:

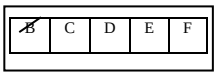
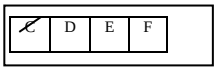
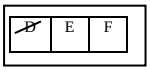
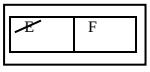


==➔



We again begin with our initial state: the node labeled A. This node is added to the stack. Note that the 1st iteration carries on from 0th, and therefore the nodes expanded. Node A is then expanded and removed from the stack , and the revealed nodes are added to the front.

stack 

v<=stack	action	visit (v)	visited sequence	u=non-visited adjacent u of v	goal test	stack update	no. of level	no. of visited nodes
v='A'	expand	A	A	u=B,C,D,E,F	n/a		0	2
v='B'	visit	B	AB	n/a	is B==goal?		1	3
v='C'	visit	C	ABC	n/a	is C==goal?		1	4
v='D'	visit	D	ABCD	n/a	is D==goal?		1	5

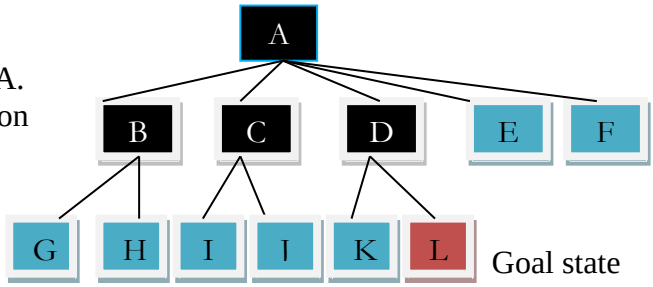
v='E'	visit	E	ABCDE	n/a	is E==goal?	~~F~~	1	6
v='F'	visit	F	ABCDEF	n/a	is F==goal?		1	7

As the stack is empty, but the goal state is not found.
So we need to begin 2nd iteration.

2nd iteration:

A ==➔

We again begin with our initial state: the node labeled A. This node is added to the stack. Note that the 2nd iteration carries on from 1st , and therefore the nodes expanded. Node A is then expanded and removed from the stack , and the revealed nodes are added to the front. Also nodes B,C,D,E,F are expanded and removed from the stack , and the revealed nodes are added to the front.



Goal state

v<=stack	action	visit (v)	visited sequence	u=non-visited adjacent u of v	goal test	stack update	no. of level	no. of visited nodes
v='A'	expand	A	A	u=B,C,D,E,F	n/a	~~B~~ C D E F	0	8
v='B'	expand	B	AB	u=G,H	n/a	~~G~~ H C D E F	1	9
v='G'	visit	G	ABG	n/a	is G==goal?	~~H~~ C D E F	2	10
v='H'	visit	H	ABGH	n/a	is H==goal?	~~C~~ D E F	2	11
v='C'	expand	C	ABGHC	u=I,J	n/a	~~I~~ J D E F	1	12
v='I'	visit	I	ABGHCI	n/a	is I==goal?	~~J~~ D E F	2	13
v='J'	visit	J	ABGHCIJ	n/a	is J==goal?	~~D~~ E F	2	14
v='D'	expand	D	ABGHCIJD	u=K,L	n/a	~~K~~ L E F	1	15
v='K'	visit	K	ABGHCIJDK	n/a	is K==goal?	~~L~~ E F	2	16

v=’L’	visit	L	ABGHCIJDKL	n/a	is L==goal?	Make stack empty	2	17



Goal State Found

Visited sequence except repeated values: ABGHCIJDKL